Curso 2026-2027
Estructura de Datos y Algoritmia
Ejercicios de costes

Departamento de Lenguajes y Sistemas Informáticos

Nota: os recuerdo que no hay que ser ni demasiado meticuloso en la realización de los ejercicios, ni demasiado hecho "a ojo". Lo que interesa es que seáis capaces de deducir la solución que tenéis al final de la sección, pero sobre todo que entendáis por qué.

Aquí tenéis un ejemplo de resolución:

Procedure Ejemplo(A: vector)

```
1     j = 1;
2     while( j < n ) do   → 1OP
3           A[j] = 2 * j; → 3 OP
4           j*=2;  → 1 OP
      end while
end procedure
```

Solución:

En la línea 2, j crece el doble cada vez →
$j = 1, 2, 4, 8 \ldots. 2^i$ (siendo i el número de iteraciones realizadas)
→ se terminará cuando j = n, es decir, $n = 2^i$ → i = log(n) iteraciones

COSTE: $1 + \log(n) * (1 + 3 + 1) + 1 = 2 + 5 * \log(n)$

COSTE ASINTÓTICO: O(log(n))

COSTE explicado: línea1 + iteraciones*(operaciones en bucle) + última ejecución de línea 2

Ejercicios

Los ejercicios tienes un nivel de dificultad diferente, utilizaré 1, 2 o 3 asteriscos indicando, en este orden menos dificultad, dificultad media, dificultad alta.

1.1. Cálculo del tiempo de ejecución

Calcula el tiempo de ejecución (obtenido en términos del número de operaciones elementales) de los siguientes algoritmos y deduce su coste asintótico:

```
----------------------------
DIFICULTAD: *
----------------------------
```

```
1: procedure A1(n:ENTERO)
2:     m = 1;
3:     q = 1;
4:     for (i = 1; i ≤ n; i++) do
5:         q = q * i;
6:         for (j = 1; j ≤ n; j++) do
7:             m = m * j;
8:         end for
9:     end for
10: end procedure
```

```
----------------------------
DIFICULTAD: *
----------------------------
```

```
1: procedure A2(n:ENTERO)
2:     m = 1; q = 1;
3:     for (i = n; i ≥ 1; i--) do
4:         q = q * i;
5:         for (j = 1; j ≤ n; j+=2) do
6:             m = m * j;
7:         end for
8:     end for
9: end procedure
```

DIFICULTAD: **

```
1: procedure A3(n:ENTERO)
2:     m = 0; i = 1;
3:     while (i ≤ n) do
4:         m = m + i;
5:         for (j = 1; j ≤ n; j*=2) do
6:             m = m * j;
7:         end for
8:         i= 2*i;
9:     end while
10: end procedure
```

DIFICULTAD: **

```
1: procedure A4(n:ENTERO)
2:     m = 0;
3:     for (i = 1; i ≤ n; i*=2) do
4:         for (j = n; j ≥ 1; j/=2) do
5:             m = m * j;
6:         end for
7:     end for
8: end procedure
```

DIFICULTAD: **

```
1: procedure A5(n:ENTERO)
2:     m = 0;
3:     for (i = 1; i ≤ n; i*=2) do
4:         for (j = 1; j ≤ n; j++) do
5:             m = m * j;
6:         end for
7:     end for
8: end procedure
```

```
----------------------------
DIFICULTAD: **
----------------------------
```

```
1: procedure A6(n:ENTERO)
2:     m = 0;
3:     for (i = 1; i ≤ n; i++) do
4:         for (j = n; j ≥ 1; j/=2) do
5:             for (k = 1; k ≤ n; k*=2) do
6:                 m = m + k;
7:             end for
8:         end for
9:     end for
10: end procedure
```

1.2. Cálculo del tiempo de ejecución con cotas

En los siguientes algoritmos el coste podrá tener un caso mejor y un caso peor. Realiza este estudio con los siguientes algoritmos:

```
----------------------------
DIFICULTAD: **
----------------------------
```

```
1: procedure A7(A:VECTOR, n:ENTERO)
2:     j = 2;
3:     if (A[j-1] >A[j]) then
4:         m = 0;
5:         for (i = 1; i ≤ n-1; i++) do
6:             m = m+i;
7:         end for
8:     else
9:         m = -1;
10:    end if
11: end procedure
```

```
----------------------------
DIFICULTAD: **
----------------------------
```

```
 1: procedure A8(n:ENTERO)
 2:     a = 0; b = n;
 3:     while (b >a) do
 4:         b = (a+b)/2;
 5:     end while
 6: end procedure
```

```
----------------------------
DIFICULTAD: **
----------------------------
```

```
 1: procedure A9(A:VECTOR, n:ENTERO)
 2:     for (i = 2; i ≤ n; i++) do
 3:         x = A[i];
 4:         j = i-1;
 5:         while (j>0 and A[j]>x) do
 6:             A[j+1] = A[j];
 7:             j = j-1;
 8:         end while
 9:         A[j+1] = x;
10:     end for
11: end procedure
```

```
----------------------------
DIFICULTAD: ***
----------------------------
```

```
 1: procedure A10(A:VECTOR, n:ENTERO, c: ENTERO)
 2:     p = 1;
 3:     q = n;
 4:     while (q ≥ p) do
 5:         i = (p+q)/2;
 6:         if (A[i]==c) then
 7:             return i;
 8:         else if (c<A[i]) then
 9:             q = i-1;
10:         else p = i+1;
11:         end if
12:     end while
13: end procedure
```

//donde A es un vector ordenado

Cotas asintóticas O de los algoritmos anteriores

Comprueba que tras calcular los costes de los algoritmos anteriores, sus cotas asintóticas coinciden con las siguientes:

	Caso mejor $\Omega(\ )$	Caso peor $O(\ )$
A1	n^2	n^2
A2	n^2	n^2
A3	$\log^2(n)$	$\log^2(n)$
A4	$\log^2(n)$	$\log^2(n)$
A5	$n\log(n)$	$n\log(n)$
A6	$n\log^2(n)$	$n\log^2(n)$
A7	1	n
A8	$\log(n)$	$\log(n)$
A9	n	n^2
A10	1	$\log(n)$

Resuelve las siguientes ecuaciones de recurrencia y da su orden de complejidad asintótica:

Ecuación	**Solución**
DIFICULTAD: * $T(n) = T(n-1) + 1$ si $n > 1$, $T(1) = 0$ en otro caso	$\Theta(n)$
DIFICULTAD: * $T(n) = T(n-1) + n$ si $n > 1$, $T(1) = 0$ en otro caso	$\Theta(n^2)$
DIFICULTAD: ** $T(n) = 2T(n/2) + n$ si $n > 4$, n potencia de 2, $T(1) = 1$ en otro caso	$\Theta(n\log(n))$
DIFICULTAD: ** $T(n) = 1 + 2T(n-1)$ si $n > 1$, $T(1) = 1$ en otro caso	$\Theta(2^n)$
DIFICULTAD: ** $T(n) = 1 + 2T(n/2)$ si $n > 1$, $T(1) = 0$ en otro caso	$\Theta(n)$
DIFICULTAD: *** $T(n) = 4T(n/2) + n^2$ si $n > 4$, n potencia de 2, $T(1) = 1$, $T(2) = 8$	$\Theta(n^2\log(n))$